# C编码规范

## 什么是一个好的程序？

- 功能实用
- 算法正确
- 代码效率高
- 可读性好

**程序可读性：程序容易读懂（不容忽视!）**

- 软件开发是团队工作，可读性在工程实践中非常重要，有利于别人接手程序并在此基础上进行改进。
- 即便是自己过段时间再看本人编写的程序进行改进，恐怕都看不懂。

# C编码规范

## 如何提高程序的可读性？

**编码的规范性和好习惯很重要！**

- 标识符命名
  - 让程序通俗易懂
- 书写格式
  - 让程序一目了然
- 注释
  - 让程序易于理解
- 良好的编程习惯
  - 事半功倍

# C编码规范——标识符命名

标识符命名：需对变量、常量、函数等标识符进行恰当的命名

命名基本要求：

- 看到标识符就能想起或猜出其含义及用途
- 名字能体现变量的类型或作用域等性质

命名注意事项：

1) 标识符名应能提供足够信息以说明其用途
   - 不能怕麻烦，少敲几个键却导致日后自己/别人读该程序需花费大量时间揣摩
   - 如果英语实在不好， 可以使用拼音
2) 全局变量取长的、描述信息多的名字；局部变量取稍短的名字
3) 可采用单词的缩写，但缩写方式要一致
   - 比如 单词 Number, 如果在某个变量里缩写成了：int nDoorNum；那么好包含 Number 单词的变量都缩写成 Num。
4) 注意使用单词的复数形式
   - 如 int nTotalStudents, nStudents ;容易让人理解成代表学生数目，而 nStudent 含义就不太明显
5) 对于返回值为真/假的函数，加 "Is" 前缀，例如：

```
int IsCanceled();
int isalpha();        // C 语言标准库函数
BOOL IsButtonPushed();
```

# C编码规范——程序书写格式

**书写格式注意事项：**

1)正确使用缩进：

- 程序一定要有缩进，否则代码的层次不明显
- 缩进要么按 Tab 键，要么按 空格键（一般 4 个），不要混用

2) 行宽与折行:

- 一行不要太长，以免阅读不便
- 太长则应折行，折行最好在运 算符前，而不要在运算符后面。例如：

```
if( Condition1() && Condition2()
      && Condition3() ) {
       DoSomething();
  }
```

3) '{', '}'位置不可随意放置

- 建议将 '{'放在一行的右边，而将 '}'单独放置一行。
- 但对于函数体或结构定义的第一个 '{'，还是单独一行更为清晰。

4) 变量和运算符之间最好加 1 个空格，例如：

```
if( nAge  >= 4 )   printf( "%d", nAge);
```

# C编码规范——注释

**必要的注释：文件开头、函数开头、全局变量定义处，等等**

工程实践中注释的写法，例如：

文件开头的注释模板如下：

```
/*****************************************************
** 文件名:
** Copyright (c) 1998-1999 *********公司技术开发部
** 创建人:
** 日　期:
** 修改人:
** 日　期:
** 描　述:
**
** 版　本:
**-------------------------------------------------------------------
---
*****************************************************
```

函数开头的注释模板如下：

```
/*****************************************************
** 函数名:
** 输　入: a,b,c
**　　a---
**　　b---
**　　c---
** 输　出: x---
**　　x 为 1, 表示...
**　　x 为 0, 表示...
** 功能描述:
** 用到的全局变量:
** 调用模块:
** 作　者:
** 日　期:
** 修　改:
** 日　期:
** 版本
*****************************************************
```

# C编码规范——良好的编程习惯

1）尽量不要用立即数，而用#define 定义成常量，便于修改。例如：

```
#define MAX_STUDENTS  20
struct SStudent aStudents [MAX_STUDENTS];
```

而不要用：  struct SStudent aStudents [20];

再例如：

```
#define TOTAL_ELEMENTS  100
for(  i = 0; i < TOTAL_ELEMENTS; i ++)   {
}
```

2）使用宏sizeof()，而不直接使用变量所占字节数的数值。例如：

应该写：

```
int nAge;
for( j = 0; j < 100; j++ )
    fwrite( fpFile, & nAge, 1, sizeof(int));
```

不应该写：

```
for( j = 0; j < 100; j++ )
    fwrite( fpFile, & nAge, 1, 4);
```

# C编码规范——良好的编程习惯

3）稍复杂的表达式要多用括号，以免优先级理解上的二义性

n = k +++ j;　　//不好

n = ( k ++ ) + j;　　//好一点

4）不容易理解的表达式应分行写

n = ( k ++ ) + j;　　应该写成：　k ++;

n = k + j;

5）嵌套的 if else 语句要多使用 { }

不好：

```
if( Condition1() )
    if( condition2()
        DoSomething();
    else
        NoCondition2();
```

应该：

```
if( Condition1() ) {
    if( condition2()
        DoSomething();
    else
        NoCondition2();
}
```

# C编码规范——良好的编程习惯

6) 单个函数的程序行数最好不超过 100 行（两个屏幕高）

7) 尽量使用标准库函数和公共函数

8) 不要随意定义全局变量，尽量使用局部变量

9) 保持注释与代码完全一致，改了代码别忘改注释

10) 循环、分支层次最好不超过五层

11) 注释可以与语句在同一行，也可以在上行

12) 一目了然的语句不加注释